Networking
Module 3&4 Assessment

S Vimal

1. Simulate a small network with switches and multiple devices. Use ping to generate traffic and observe the MAC address table of the switch. Capture packets using Wireshark to analyze Ethernet frames and MAC addressing.

2. Capture and analyze Ethernet frames using Wireshark. Inspect the structure of the frame, including destination and source MAC addresses, Ethertype, payload, and FCS. Use GNS3 or Packet Tracer to simulate network traffic.

3. Configure static IP addresses, modify MAC addresses, and verify networkping and ifconfig commands.

4. Troubleshoot Ethernet Communication with ping and traceroute -> Using cisco packet tracer:

5. Create a simple LAN setup with two Linux machines connected via a switch.

6. Ping from one machine to the other. If it fails, use ifconfig to ensure the IP addresses are configured correctly.

7. Use traceroute to identify where the packets are being dropped if the ping fails.

8. Research the Linux kernel's handling Of Ethernet devices and network interfaces. Write a short report on how the Linux kernel supports Ethernet communication (referencing kernel.org documentation).

9. Describe how you would configure a basic LAN interface using the ip command in Linux (kernel.org).

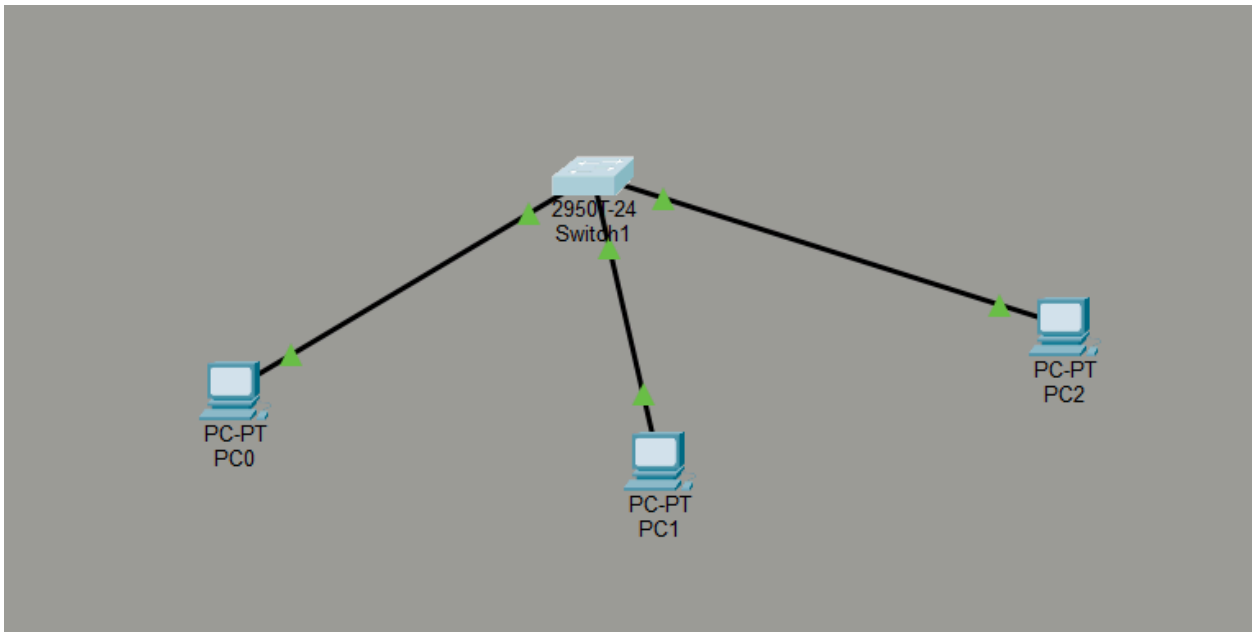
10. Use Linux to view the MAC address table of a switch (if using a Linux-based network switch). Use the bridge or ip link commands to inspect the MAC table and demonstrate a basic switch's operation.

11. Submit your findings with explanations of what you observe.

Solutions

1.

Creating a small network using cisco packet tracer



Empty MAC Address table before sending any packets within network

```
Switch>
Switch>en
Switch#enable
Switch#show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
```

Ping one device to another

```
C:\>ping 192.62.0.20

Pinging 192.62.0.20 with 32 bytes of data:

Reply from 192.62.0.20: bytes=32 time<1ms TTL=128
Reply from 192.62.0.20: bytes=32 time<1ms TTL=128
Reply from 192.62.0.20: bytes=32 time<1ms TTL=128
Reply from 192.62.0.20: bytes=32 time<1ms TTL=128

Ping statistics for 192.62.0.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Now the switch updates the MAC table after learning about devices in the network after flooding them

```
Switch#show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0001.6329.1db1    DYNAMIC     Fa0/1
   1    0060.2fb9.7d0e    DYNAMIC     Fa0/2
```

2.
Structure of a Frame

```
▾ Frame 8: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface enp0s3, id 0
    Section number: 1
  ▸ Interface id: 0 (enp0s3)
    Encapsulation type: Ethernet (1)
    Arrival Time: Mar  9, 2026 23:43:08.610372991 IST
    UTC Arrival Time: Mar  9, 2026 18:13:08.610372991 UTC
    Epoch Arrival Time: 1773079988.610372991
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.000024413 seconds]
    [Time delta from previous displayed frame: 0.000024413 seconds]
    [Time since reference or first frame: 3.213687644 seconds]
    Frame Number: 8
    Frame Length: 98 bytes (784 bits)
    Capture Length: 98 bytes (784 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:icmp:data]
    [Coloring Rule Name: ICMP]
```

Ethertype, Source and Destination MAC Address

```
▾ Ethernet II, Src: PCSSystemtec_f5:68:17 (08:00:27:f5:68:17), Dst: PCSSystemtec_9b:26:bf (08:00:27:9b
  ▾ Destination: PCSSystemtec_9b:26:bf (08:00:27:9b:26:bf)
      Address: PCSSystemtec_9b:26:bf (08:00:27:9b:26:bf)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  ▾ Source: PCSSystemtec_f5:68:17 (08:00:27:f5:68:17)
      Address: PCSSystemtec_f5:68:17 (08:00:27:f5:68:17)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
```

Checksum

```
▾ Internet Control Message Protocol
    Type: 0 (Echo (ping) reply)
    Code: 0
    Checksum: 0x3d17 [correct]
    [Checksum Status: Good]
    Identifier (BE): 4842 (0x12ea)
    Identifier (LE): 59922 (0xea12)
    Sequence Number (BE): 4 (0x0004)
    Sequence Number (LE): 1024 (0x0400)
    [Request frame: 7]
    [Response time: 0.024 ms]
    Timestamp from icmp data: Mar  9, 2026 23:43:08.700547000 IST
    [Timestamp from icmp data (relative): -0.090174009 seconds]
```

Payload

```
▸ Frame 8: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface enp0s3, id 0
▸ Ethernet II, Src: PCSSystemtec_f5:68:17 (08:00:27:f5:68:17), Dst: PCSSystemtec_9b:26:bf (08:00:27:9b
▸ Internet Protocol Version 4, Src: 192.168.1.50, Dst: 192.168.1.20
▸ Internet Control Message Protocol

▾ Data (40 bytes)
    Data: 101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f3031323334353637
    [Length: 40]
```

3.

Static IP can be configured using “ip addr add” command, and is set up and verified using set and show command

```
vimal@vimal-VirtualBox:~/networking$ sudo ip addr add 192.168.1.20/24 dev enp0s3
vimal@vimal-VirtualBox:~/networking$ sudo ip link set enp0s3 up
vimal@vimal-VirtualBox:~/networking$ ip adddr show enp0s3 up
Object "adddr" is unknown, try "ip help".
vimal@vimal-VirtualBox:~/networking$ ip addr show enp0s3 up
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 08:00:27:f5:68:17 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.20/24 scope global enp0s3
       valid_lft forever preferred_lft forever
```

MAC address is changed using set enp0s3 address command and is verified using ip link show enp0s3

```
vimal@vimal-VirtualBox:~/networking$ sudo ip link set enp0s3 down
vimal@vimal-VirtualBox:~/networking$ sudo ip link set enp0s3 address 02:00:00:00
:00:01
vimal@vimal-VirtualBox:~/networking$ sudo ip link set enp0s3 up
vimal@vimal-VirtualBox:~/networking$ ip link show enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
mode DEFAULT group default qlen 1000
    link/ether 02:00:00:00:00:01 brd ff:ff:ff:ff:ff:ff permaddr 08:00:27:f5:68:1
7
```

Ifconfig command

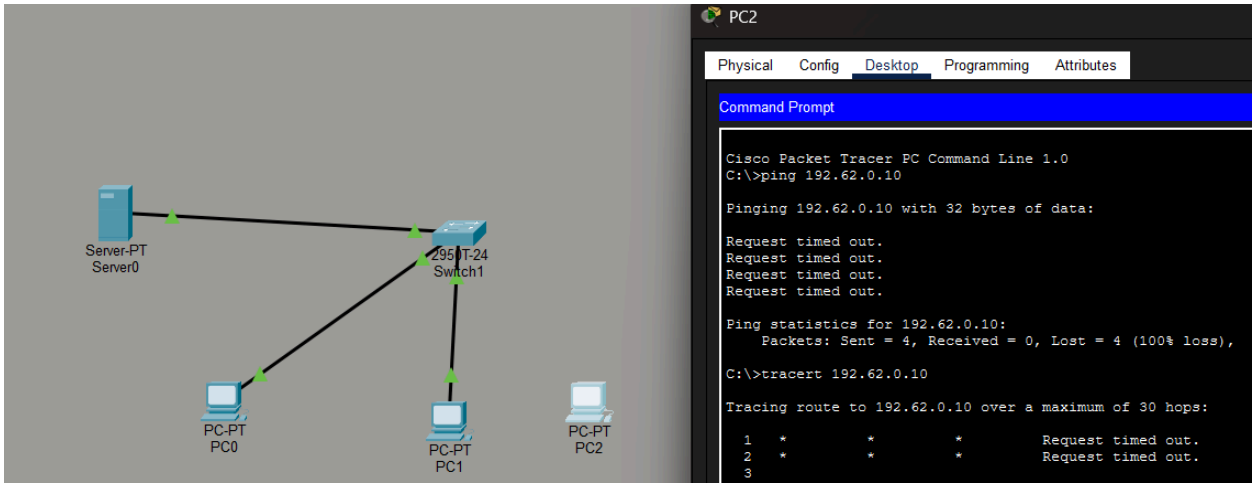
```
vimal@vimal-VirtualBox:~/networking$ ifconfig enp0s3
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.20  netmask 255.255.255.0  broadcast 0.0.0.0
        ether 02:00:00:00:00:01  txqueuelen 1000  (Ethernet)
        RX packets 1  bytes 87 (87.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 160  bytes 24491 (24.4 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Verifying connection using another VM machine to the current machine's IP

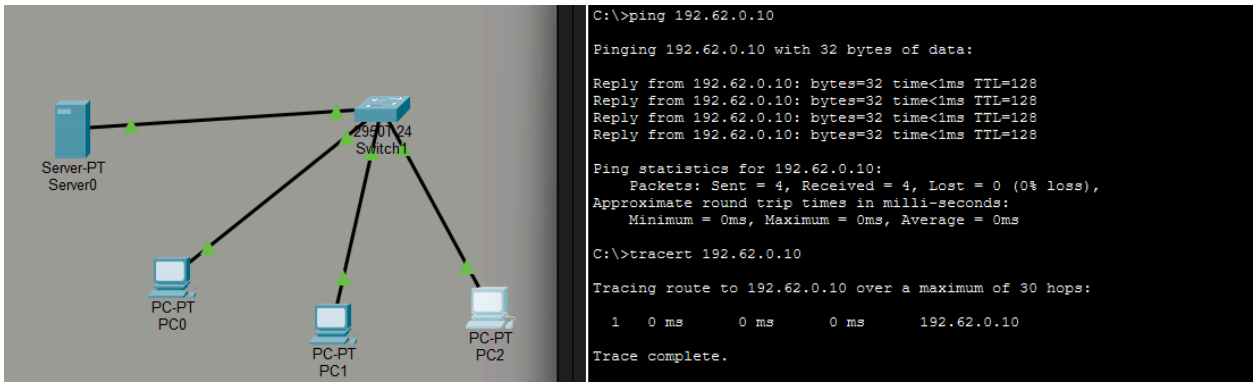
```
student@administrator-VirtualBox:~$ ping 192.168.1.20
PING 192.168.1.20 (192.168.1.20) 56(84) bytes of data.
64 bytes from 192.168.1.20: icmp_seq=1 ttl=64 time=1.69 ms
64 bytes from 192.168.1.20: icmp_seq=2 ttl=64 time=0.857 ms
64 bytes from 192.168.1.20: icmp_seq=3 ttl=64 time=0.913 ms
64 bytes from 192.168.1.20: icmp_seq=4 ttl=64 time=1.08 ms
64 bytes from 192.168.1.20: icmp_seq=5 ttl=64 time=0.812 ms
64 bytes from 192.168.1.20: icmp_seq=6 ttl=64 time=1.27 ms
64 bytes from 192.168.1.20: icmp_seq=7 ttl=64 time=1.49 ms
^C
--- 192.168.1.20 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6393ms
rtt min/avg/max/mdev = 0.812/1.156/1.685/0.310 ms
```

4.

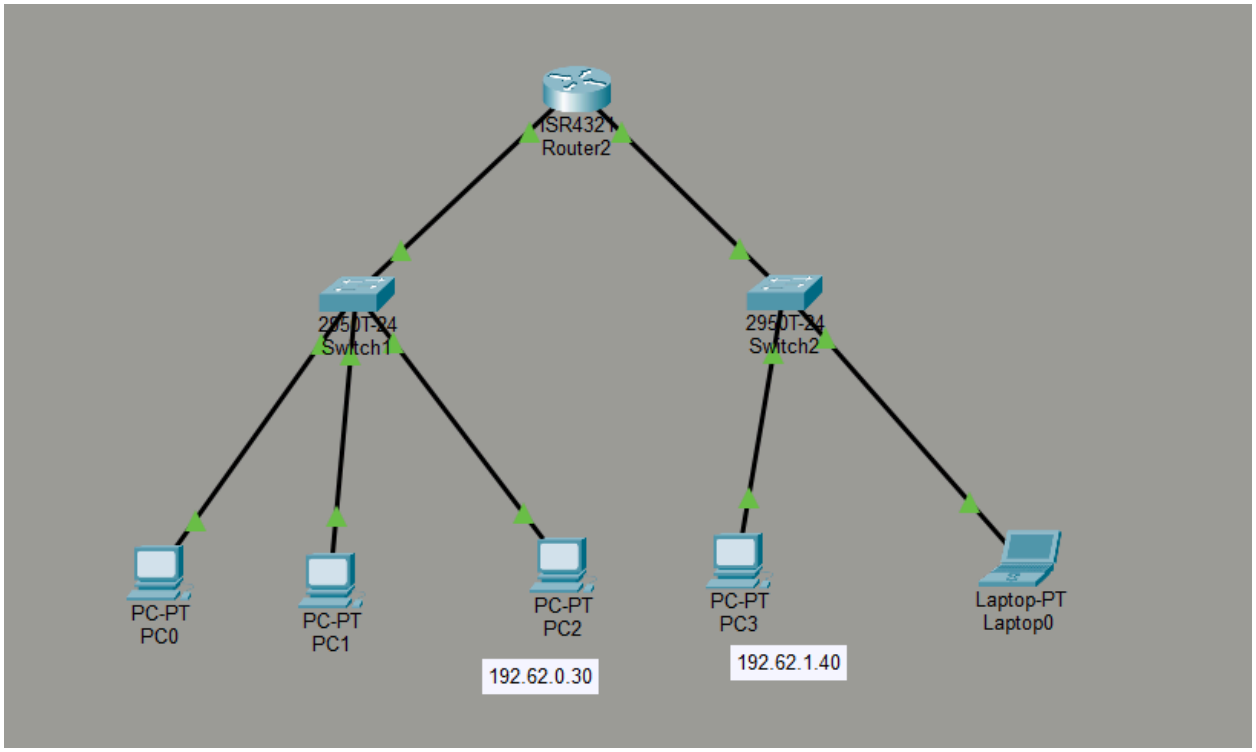
Using network commands to check connectivity of the LAN



This happens because the system is not connected to the switch, which connects to the rest of the nodes. After connecting with the switch, the results can be seen



While pinging from PC2 to PC3 (different LANs), it can be seen the network is not established, even though cables are connected




```
C:\>ping 192.62.1.40

Pinging 192.62.1.40 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.62.1.40:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>tracert 192.62.1.40

Tracing route to 192.62.1.40 over a maximum of 30 hops:

  1   *        *        *        Request timed out.
  2   *        *        *        Request timed out.
  3   *
```

This can be troubleshooted by configuring the interface of router with default gateways of each networks from respective switches.

```
Router(config)#interface GigabitEthernet0/0/0
Router(config-if)#ip address 192.62.0.1 255.255.255.0
Router(config-if)#ip address 192.62.0.1 255.255.255.0
Router(config-if)#
Router(config-if)#
Router(config-if)#exit
Router(config)#interface GigabitEthernet0/0/1
Router(config-if)#
Router(config-if)#exit
Router(config)#interface GigabitEthernet0/0/0
Router(config-if)#
Router(config-if)#exit
Router(config)#interface GigabitEthernet0/0/1
Router(config-if)#ip address 192.62.1.1 255.255.255.0
Router(config-if)#
Router(config-if)#
Router(config-if)#exit
```

```
C:\>ping 192.62.1.40

Pinging 192.62.1.40 with 32 bytes of data:

Request timed out.
Reply from 192.62.1.40: bytes=32 time<1ms TTL=127
Reply from 192.62.1.40: bytes=32 time<1ms TTL=127
Reply from 192.62.1.40: bytes=32 time<1ms TTL=127

Ping statistics for 192.62.1.40:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>tracert 192.62.1.40

Tracing route to 192.62.1.40 over a maximum of 30 hops:

  1   0 ms      0 ms      0 ms      192.62.0.1
  2   0 ms      0 ms      0 ms      192.62.1.40

Trace complete.
```

5.

To create a simple LAN setup with two Linux machines connected via a switch, we need to configure the virtual hardware in Virtual Box. By changing the network adapter settings of both virtual machines to an "Internal Network" (or "LAN Segment") and assigning them the exact same network name, we are essentially plugging both machines into the same isolated, virtual network switch. This creates a closed local area

network (LAN) where the two machines can freely pass Ethernet frames back and forth without outside interference.

Once the virtual cables are connected to the switch, we must configure the software side by assigning static IP addresses within the same subnet so the operating systems know how to reach each other.

On the first Linux machine, open the terminal and assign an address while bringing the interface online by running

sudo ip addr add 192.168.1.10/24 dev enp0s3

sudo ip link set dev enp0s3 up.

Then, on the second machine, place it on the exact same network by assigning a different host address, using

sudo ip addr add 192.168.1.20/24 dev enp0s3

sudo ip link set dev enp0s3 up.

This completes the LAN setup, leaving both machines fully configured and ready to communicate.

```
student@administrator-VirtualBox:~/Desktop$ sudo ip addr add 192.168.1.10/24 dev
 enp0s3
[sudo] password for student:
student@administrator-VirtualBox:~/Desktop$ sudo ip link set dev enp0s3 up
student@administrator-VirtualBox:~/Desktop$
```

```
vimal@vimal-VirtualBox:~/Desktop$ sudo ip addr add 192.168.1.20/24 dev enp0s3
vimal@vimal-VirtualBox:~/Desktop$ sudo ip link set dev enp0s3 up
vimal@vimal-VirtualBox:~/Desktop$
```

6.

Using ping to verify the connectivity and troubleshoot if any errors are found

```
vimal@vimal-VirtualBox:~/Desktop$ ping 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=2.75 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=1.12 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=64 time=1.26 ms
64 bytes from 192.168.1.10: icmp_seq=4 ttl=64 time=1.89 ms
64 bytes from 192.168.1.10: icmp_seq=5 ttl=64 time=0.871 ms
64 bytes from 192.168.1.10: icmp_seq=6 ttl=64 time=1.05 ms
64 bytes from 192.168.1.10: icmp_seq=7 ttl=64 time=2.03 ms
64 bytes from 192.168.1.10: icmp_seq=8 ttl=64 time=1.08 ms
64 bytes from 192.168.1.10: icmp_seq=9 ttl=64 time=1.17 ms
^C
--- 192.168.1.10 ping statistics ---
9 packets transmitted, 9 received, 0% packet loss, time 8014ms
rtt min/avg/max/mdev = 0.871/1.468/2.751/0.584 ms
```

```
student@administrator-VirtualBox:~/Desktop$ ping 192.168.1.20
PING 192.168.1.20 (192.168.1.20) 56(84) bytes of data.
64 bytes from 192.168.1.20: icmp_seq=1 ttl=64 time=2.21 ms
64 bytes from 192.168.1.20: icmp_seq=2 ttl=64 time=1.04 ms
64 bytes from 192.168.1.20: icmp_seq=3 ttl=64 time=0.961 ms
64 bytes from 192.168.1.20: icmp_seq=4 ttl=64 time=0.866 ms
^C
--- 192.168.1.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3248ms
rtt min/avg/max/mdev = 0.866/1.267/2.207/0.545 ms
```

7.

Here since the network is properly connected, traceroute shows the hop to the other machine, we can try this with setting one of the link down.

```
vimal@vimal-VirtualBox:~/Desktop$ traceroute 192.168.1.10
traceroute to 192.168.1.10 (192.168.1.10), 64 hops max
  1   192.168.1.10  1.564ms  0.004ms  0.673ms
```

Setting link down on one machine

student@administrator-VirtualBox:~/Desktop$ sudo ip link set dev enp0s3 down

Pinging and Traceroute while network is down

```
vimal@vimal-VirtualBox:~/Desktop$ ping 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
From 192.168.1.20 icmp_seq=9 Destination Host Unreachable
From 192.168.1.20 icmp_seq=10 Destination Host Unreachable
From 192.168.1.20 icmp_seq=11 Destination Host Unreachable
From 192.168.1.20 icmp_seq=12 Destination Host Unreachable
From 192.168.1.20 icmp_seq=13 Destination Host Unreachable
From 192.168.1.20 icmp_seq=14 Destination Host Unreachable
^C
--- 192.168.1.10 ping statistics ---
17 packets transmitted, 0 received, +6 errors, 100% packet loss, time 16388ms
pipe 4
vimal@vimal-VirtualBox:~/Desktop$ traceroute 192.168.1.10
traceroute to 192.168.1.10 (192.168.1.10), 64 hops max
  1   *  192.168.1.20  100.508ms !H  0.004ms !H
```

When executing traceroute 192.168.1.10 after a simulated ping failure, the output displayed !H (Host Unreachable) at hop 1, returning the source machine's own IP address (192.168.1.20). This identifies that the packets are being dropped immediately at the source interface. Because the machines are on the same LAN segment, the source machine failed to resolve the target's MAC address via ARP, causing the local kernel to drop the packet before it even traversed the virtual switch.

8.

To handle Ethernet communication, the Linux kernel abstracts physical network hardware into logical software structures. This allows the operating system's network stack to process packets uniformly, regardless of whether the physical network card is made by Intel, Realtek, or Broadcom. According to kernel.org networking documentation, this process relies on three massive core components: the net_device structure, the sk_buff structure, and NAPI.

1. The net_device Structure

Every network interface in Linux (such as eth0, enp0s3, or lo) is represented internally by a C structure called net_device. This structure is the ultimate rulebook for the interface. It stores the device's MAC address, MTU (Maximum Transmission Unit), interface state (UP/DOWN), and function pointers that tell the kernel exactly which device driver to use to transmit raw Ethernet frames out of the physical port.

2. The Socket Buffer (sk_buff)

When an Ethernet frame arrives at the network card, the kernel needs a way to move it up to the application (like a web browser) without constantly copying the data, which would waste CPU memory. It does this using the sk_buff (Socket Buffer).

The sk_buff is the fundamental data structure for networking in Linux. It acts like a metadata wrapper around the packet. As the packet travels up the OSI model (from Ethernet Layer 2 -> IP Layer 3 -> TCP Layer 4), the kernel simply moves pointers within the sk_buff to strip off the headers, rather than moving the actual payload data.

3. NAPI (New API) and Packet Reception

Historically, every time an Ethernet frame arrived, the network card sent a hardware interrupt to the CPU. However, on modern high-speed networks (like Gigabit or 10-Gigabit Ethernet), thousands of frames arrive per second. Triggering an interrupt for every single frame would cause an "interrupt storm," freezing the entire computer. To solve this, the Linux kernel uses **NAPI (New API)**. When the first packet arrives, an interrupt is triggered. The kernel then disables further interrupts and switches to a "polling" mode. It rapidly scoops up batches of Ethernet frames directly from the network card's memory ring-buffer all at once. Once the buffer is empty, it turns interrupts back on.

9.

In order to setup a basic Local Area Network (LAN) interface in Linux one uses the ip command suite to communicate with the networking stack of the kernel directly.

To start with, you need to query the kernel to determine the logical name of your network interface card (NIC) e.g. eth0 or enp0s3. After the identification of the interface, you give the ip address add command to provide a unique IPv4 address and a subnet mask to that particular device. One of the most important aspects of LAN configuration is the subnet mask (often expressed in a CIDR notation, such as /24), which determines the limits of your local network, and informs the Linux kernel of what IP addresses directly it can communicate with without having to route the packet through a router.

The binding of the IP address to the interface may disable the device administratively. To correct this, you apply the ip link set command that will change the state of the

interface to be up instead of down. This command is used to order the kernel to boot the interface and start receiving and sending raw Ethernet packets onto the LAN. You finally check the configuration by inspecting the details in the interface to confirm that the IP address is added, the administrative state is set to UP and the physical virtual cable is actually detected (the LOWER_UP flag indicates that). Since the ip command changes the live configuration of the kernel in the RAM, this allows real time LAN communication but will restore back to its original state once the machine is rebooted.

Adding a new IP address for the VM1

```
vimal@vimal-VirtualBox:~/Desktop$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAU
 group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state U
mode DEFAULT group default qlen 1000
    link/ether 08:00:27:f5:68:17 brd ff:ff:ff:ff:ff:ff
vimal@vimal-VirtualBox:~/Desktop$ sudo ip addr add 192.168.1.50/24 dev enp0s3
[sudo] password for vimal:
vimal@vimal-VirtualBox:~/Desktop$ sudo ip link set eth0 up
Cannot find device "eth0"
vimal@vimal-VirtualBox:~/Desktop$ sudo ip link set enp0s3 up
```

Pinging from other VM in the LAN to check whether device to verify connection

```
student@administrator-VirtualBox:~/networking$ ping 192.168.1.50
PING 192.168.1.50 (192.168.1.50) 56(84) bytes of data.
64 bytes from 192.168.1.50: icmp_seq=1 ttl=64 time=1.52 ms
64 bytes from 192.168.1.50: icmp_seq=2 ttl=64 time=0.978 ms
64 bytes from 192.168.1.50: icmp_seq=3 ttl=64 time=0.846 ms
64 bytes from 192.168.1.50: icmp_seq=4 ttl=64 time=1.28 ms
64 bytes from 192.168.1.50: icmp_seq=5 ttl=64 time=0.842 ms
64 bytes from 192.168.1.50: icmp_seq=6 ttl=64 time=0.863 ms
^C
--- 192.168.1.50 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5530ms
rtt min/avg/max/mdev = 0.842/1.055/1.523/0.258 ms
```

10.

Linux can act as a Layer 2 switch using the kernel's bridge module. This is used in virtual environments (KVM/QEMU, Docker, GNS3) and on embedded network appliances.

First creating a new bridge interface, named it as br0 and turn it on. **And then plugging in the NIC enp0s3**

```
student@administrator-VirtualBox:~/Desktop$ sudo ip link add name br0 type bridge
student@administrator-VirtualBox:~/Desktop$ sudo ip link set dev enp0s3 master br0
```

Providing the bridge with an IP address , since enp0s3 is now a port, the switch br0 needs an IP address so it can communicate on the network. And then setting up or turning on the bridge and port.

```
student@administrator-VirtualBox:~/Desktop$ sudo ip addr add 192.168.1.10/24 dev br0
student@administrator-VirtualBox:~/Desktop$ sudo ip link set dev br0 up
student@administrator-VirtualBox:~/Desktop$ sudo ip link set dev enp0s3 up
```

To generate traffic pinging from another machine.

```
vimal@vimal-VirtualBox:~/Desktop$ ping -c 4 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=1.40 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=0.877 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=64 time=1.12 ms
64 bytes from 192.168.1.10: icmp_seq=4 ttl=64 time=1.76 ms

--- 192.168.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 0.877/1.289/1.763/0.330 ms
```

Now using ***bridge fdb show br br0*** command to check the MAC address

```
student@administrator-VirtualBox:~/Desktop$ bridge fdb show br br0
08:00:27:f5:68:17 dev enp0s3 master br0
08:00:27:9b:26:bf dev enp0s3 vlan 1 master br0 permanent
08:00:27:9b:26:bf dev enp0s3 master br0 permanent
01:00:5e:00:00:01 dev enp0s3 self permanent
33:33:00:00:00:01 dev enp0s3 self permanent
33:33:00:00:00:01 dev br0 self permanent
01:00:5e:00:00:6a dev br0 self permanent
33:33:00:00:00:6a dev br0 self permanent
01:00:5e:00:00:01 dev br0 self permanent
33:33:ff:f5:cc:77 dev br0 self permanent
01:00:5e:00:00:fb dev br0 self permanent
33:33:00:00:00:fb dev br0 self permanent
82:79:27:f5:cc:77 dev br0 vlan 1 master br0 permanent
82:79:27:f5:cc:77 dev br0 master br0 permanent
```

Here 08:00:27:f5:68:17 is the MAC address of the machine from which we pinged.

Analyzing the output:

08:00:27:f5:68:17 dev enp0s3 master br0: When VM2 (08:00:27:f5:68:17) sent a ping request, the frame entered the enp0s3 port. The Linux kernel inspected the incoming Ethernet frame, read the Source MAC address, and automatically mapped it to the enp0s3 port in its Forwarding Database (FDB).

08:00:27:9b:26:bf dev enp0s3 master br0 permanent: This is the physical MAC address of the VM1 network card itself (enp0s3). The state is listed as permanent because this MAC address is physically bound to that local interface. The switch knows it will never have to forward frames for this MAC address out to the network, because it belongs to the switch itself.